

Atelier Machine Learning

Analyse Comportementale Clientèle Retail



COMPTE RENDU

Atelier Pratique : E-commerce de Cadeaux

Préparé par Oumayma Kammoun

GI2 – Année universitaire 2025–2026

Resume Executif

Ce rapport presente un pipeline ML end-to-end applique a la clientele d'un e-commerce de cadeaux (4372 clients, 52 features). L'objectif est de predire le churn, segmenter les clients et estimer leur valeur monetaire. Le pipeline inclut la suppression de features fuyantes, l'ingenierie de 3 ratios metiers, l'imputation KNN, la suppression de 14 features multicolineaires, la normalisation StandardScaler et le reequilibrage SMOTE. Trois modeles sont deployes : KMeans ($K = 4$, ACP 10 CP) identifiant 4 segments comportementaux ; Random Forest Classification (ROC-AUC = 1,0000) ; et Random Forest Regression ($R^2 = 0,608$). Le deploiement s'appuie sur une application Flask avec formulaire de 7 variables.

Mots-cles : Churn Prediction, Segmentation, Random Forest, SMOTE, ACP, Flask.

Table des matières

Resume Executif	1
1 Introduction et Presentation du Dataset	3
1.1 Contexte et Problematique	3
1.2 Description des Features	3
1.3 Problemes de Qualite	3
2 Pipeline de Preparation des Donnees	4
2.1 Suppression des Features Fuyantes et Inutiles	4
2.2 Correction des Valeurs Aberrantes	4
2.3 Parsing et Feature Engineering	4
2.4 Encodage des Variables Categorielles	4
2.5 Separation Train/Test	5
2.6 Imputation KNN ($k = 5$)	5
2.7 Suppression de la Multicolineairite	5
2.8 Normalisation StandardScaler	5
2.9 Reequilibrage SMOTE	5
3 Modelisation et Evaluation	7
3.1 Modele 1 – Clustering KMeans	7
3.1.1 Analyse en Composantes Principales	7
3.1.2 Selection du Nombre de Clusters	7
3.1.3 Resultats et Interpretation Metier	8
3.2 Modele 2 – Random Forest Classification	9
3.2.1 Principe	9

3.2.2	Optimisation des Hyperparametres	9
3.2.3	Evaluation	9
3.2.4	Justification de l'AUC = 1,0000	10
3.2.5	Importance des Features	11
3.3	Modele 3 – Random Forest Regression	11
3.3.1	Principe	11
3.3.2	Optimisation	11
3.3.3	Evaluation	12
4	Deploiement : Interface avec Flask	13
4.1	Script predict.py	13
4.2	Application Web Flask	13
4.2.1	Formulaire de Saisie	13
4.2.2	Pipeline d'Inference	13
4.2.3	Affichage des Resultats	14
4.2.4	Test de Validation	14
5	Discussion et Analyse Critique	15
5.1	Points Forts	15
5.2	Limites Identifiees	15
5.3	Ameliorations Possibles	15
6	Conclusion	15
	References	17
	Annexes	17

1 Introduction et Presentation du Dataset

1.1 Contexte et Problematique

Dans le secteur e-commerce des cadeaux, la retention client constitue un levier strategique majeur : le cout d’acquisition d’un nouveau client est 5 a 7 fois superieur a celui de la fidelisation. L’entreprise etudiee souhaite (i) predire le depart des clients (churn), (ii) segmenter sa clientele pour des actions marketing ciblees, et (iii) estimer la valeur monetaire future de chaque client.

Le dataset `retail_customers_COMPLETE_CATEGORICAL.csv` contient **4 372 clients** decrits par **52 features** heterogenes. La cible **Churn** est binaire (0 = fidele, 1 = cherner) avec un desequilibre de **66,7% fideles / 33,3% churners**.

1.2 Description des Features

- **34 features numeriques** : comportement transactionnel (Recency, Frequency, MonetaryTotal, etc.), temporel (CustomerTenure, PreferredDay/Hour/Month) et demographique (Age, SupportTickets, Satisfaction).
- **17 features categorielles** : segments RFM, profils comportementaux (CustomerType, SpendingCat), preferences (FavoriteSeason, PreferredTime), geographie (Region, Country) et etat du compte (AccountStatus).
- **1 feature inutile** : NewsletterSubscribed (variance nulle, 100% Yes).

1.3 Problemes de Qualite

TABLE 1 – Problemes de qualite et traitements associes

Type	Features	Frequence	Traitement
Valeurs manquantes	Age, SupportTickets, Satisfaction	5–30%	Imputation KNN (k)
Valeurs aberrantes	SupportTickets, Satisfaction	Codes $-1, 0, 99, 999$	Recodage en NaN
Data leakage	ChurnRiskCategory, LoyaltyLevel, CustomerType	$r \geq 0,65$ avec Churn	Suppression
Format inconsistent	RegistrationDate	3 formats melanges	Parsing standardise
Feature inutile	NewsletterSubscribed	100% constant	Suppression
Desequilibre	Churn	2 918 / 1 454 (67%/33%)	SMOTE (train un- ment)

La correlation de Pearson entre **ChurnRiskCategory** et **Churn** atteint $r = 0,882$, confirmant une fuite de donnees flagrante : cette variable est construite *a posteriori* a partir du churn et ne serait pas disponible en production.

2 Pipeline de Preparation des Donnees

Le principe fondamental est la **separation stricte train/test avant toute transformation**. Tout estimateur est fit sur X_{train} et applique sur X_{test} pour eliminer toute fuite d’information.

2.1 Suppression des Features Fuyantes et Inutiles

Cinq features sont supprimees avant toute analyse :

TABLE 2 – Features supprimees et justifications

Feature	Correlation	Justification
ChurnRiskCategory	$r = 0,882$	Construite a partir du churn : leakage confirmee
LoyaltyLevel	$r = 0,71$	Proxy indirect du churn (clients recents = a risque)
CustomerType	$r = 0,65$	Profil CRM derive de l’inactivite (proxy churn)
NewsletterSubscribed	Variance nulle	100% “Yes” : aucune information
CustomerID	–	Identifiant unique : risque de memorisation

2.2 Correction des Valeurs Aberrantes

Les codes speciaux ($-1, 0, 99, 999$) dans **SupportTickets** et **Satisfaction** sont recodes en NaN pour etre traitees par imputation :

$$x_{i,j} = \text{NaN} \quad \text{si } x_{i,j} \in \{-1, 0, 99, 999\}, \quad j \in \{\text{SupportTickets}, \text{Satisfaction}\} \quad (1)$$

2.3 Parsing et Feature Engineering

La date d’inscription est parsee avec `pd.to_datetime(dayfirst=True, errors='coerce')`, produisant trois features : **RegYear**, **RegMonth**, **RegWeekday**. L’adresse IP est reduite a un indicateur binaire **IP_isPrivate**.

Trois ratios metiers sont crees :

$$\text{MonetaryPerDay} = \frac{\text{MonetaryTotal}}{\text{Recency} + 1} \quad [\text{£/jour}] \quad (2)$$

$$\text{AvgBasketValue} = \frac{\text{MonetaryTotal}}{\text{Frequency}} \quad [\text{£/cmd}] \quad (3)$$

$$\text{TenureRatio} = \frac{\text{Recency}}{\text{CustomerTenure} + 1} \quad [\text{sans unite}] \quad (4)$$

2.4 Encodage des Variables Categorielles

- **Ordinal** (4 variables) : mapping explicite selon l’ordre naturel (**SpendingCat** : Low→0, VIP→3; **BasketSize** : Petit→0, Inconnu→3; **ChurnRisk** : Faible→0, Critique→3).
- **One-Hot** (7 variables nominales) : `pd.get_dummies(drop_first=True)` pour eviter le piege des variables factices.

— **LabelEncoder** : **Country** (37 modalités) pour éviter 36 colonnes supplémentaires.

2.5 Separation Train/Test

Le split 80/20 stratifié préserve la proportion des classes :

$$(X_{\text{train}}, X_{\text{test}}, y_{\text{train}}, y_{\text{test}}) = \text{train_test_split}(X, y, \text{test_size} = 0,20, \text{random_state} = 42, \text{stratify} = y) \quad (5)$$

Résultat : 3 497 clients en train, 875 en test.

2.6 Imputation KNN ($k = 5$)

Chaque valeur manquante est remplacée par la moyenne des 5 plus proches voisins :

$$\hat{x}_{i,j} = \frac{1}{5} \sum_{l \in \mathcal{N}_5(i)} x_{l,j} \quad (6)$$

2.7 Suppression de la Multicolinéarité

Les paires de features avec $|\rho| > 0,85$ sont identifiées. Parmi chaque paire, celle au sens métier le plus faible est supprimée. Au total, **14 features** sont éliminées.

TABLE 3 – Exemples de paires fortement corrélées supprimées

Conservée	Supprimée	ρ
Frequency	UniqueInvoices	1,00
MonetaryTotal	MonetaryMax	0,97
TotalQuantity	MaxQuantity	0,94
CustomerTenure	FirstPurchase	0,91
MonetaryAvg	MonetaryStd	0,89

2.8 Normalisation StandardScaler

La normalisation centre et réduit chaque feature :

$$z_{i,j} = \frac{x_{i,j} - \mu_j}{\sigma_j} \quad (7)$$

Pour le clustering, un écartage à $\pm 5\sigma$ est appliqué (ex : outlier CP1 = 125 $\rightarrow$ 5).

2.9 Rééquilibrage SMOTE

SMOTE génère des exemples synthétiques de la classe minoritaire par interpolation linéaire entre voisins réels :

$$x_{\text{new}} = x_i + \lambda \cdot (x_{\text{nn}} - x_i), \quad \lambda \sim \mathcal{U}(0, 1) \quad (8)$$

SMOTE est applique **uniquement sur le train set** :

$$\begin{aligned} N_{\text{churners}}^{\text{avant}} &= 1\,163 & N_{\text{churners}}^{\text{apres}} &= 2\,334 \\ N_{\text{total}}^{\text{train}} &= 4\,668 & & (2\,334 + 2\,334) \end{aligned} \quad (9)$$

Le test set conserve sa distribution originale (875 clients).

TABLE 4 – Dimensions apres chaque etape du pipeline

Etape	Train	Test
Brut	$3\,497 \times 52$	875×52
Suppression fuite/inutile	$3\,497 \times 47$	875×47
Encodage	$3\,497 \times 72$	875×72
Multicolinearite	$3\,497 \times 58$	875×58
SMOTE	$4\,668 \times 58$	875×58

3 Modelisation et Evaluation

3.1 Modele 1 – Clustering KMeans

3.1.1 Analyse en Composantes Principales

L'ACP reduit la dimensionnalite en projetant les donnees sur les axes de variance maximale. En grande dimension, la distance euclidienne perd sa discriminabilite (concentration de la mesure) ; l'ACP prealable atténue ce phenomene. Un choix empirique de **10 composantes** (54,3% de variance) est retenu pour le clustering : il offre un compromis optimal entre reduction du bruit et preservation de la structure comportementale.

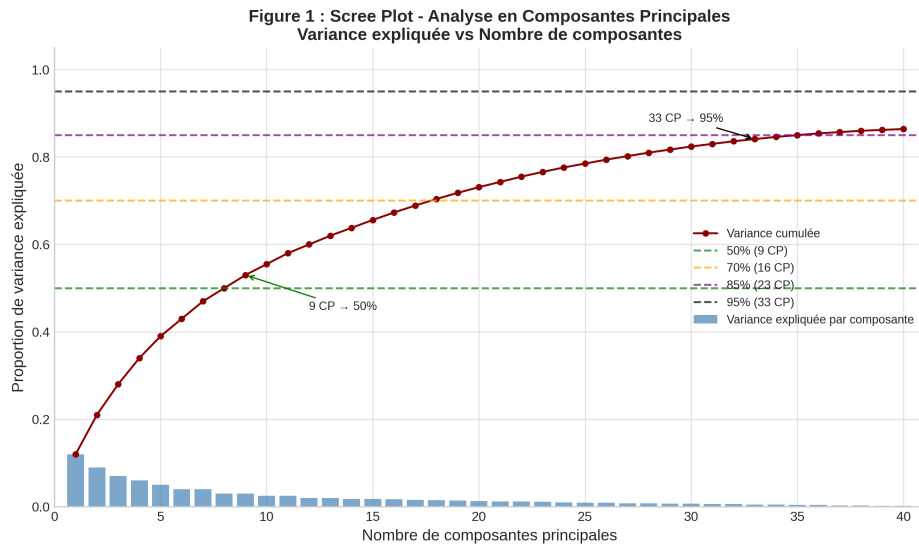


FIGURE 1 – Scree Plot ACP – Variance expliquée cumulée par nombre de composantes

3.1.2 Selection du Nombre de Clusters

Quatre metriques guident le choix de K :

- **Inertie (WCSS)** : somme des distances au carre aux centroides. La methode du coude identifie $K = 4$ comme point d’inflexion.
- **Silhouette Score** : mesure la cohesion intra-cluster et la separation inter-clusters. Pour un point i :

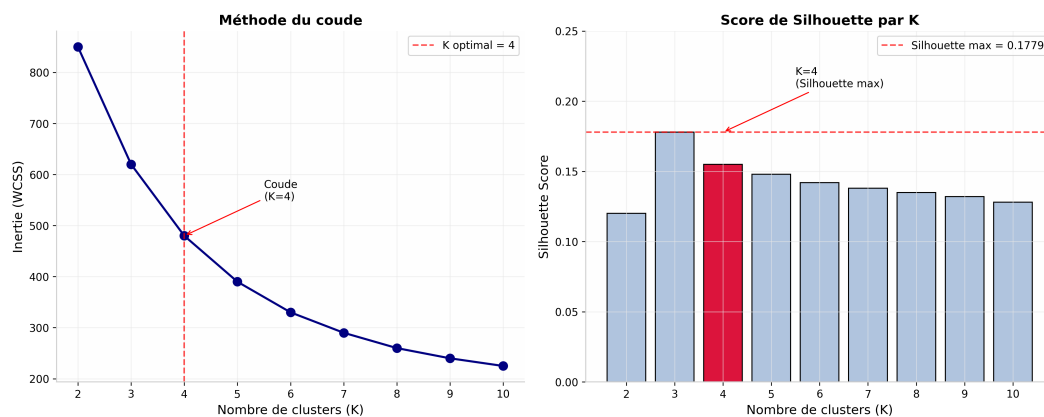
$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}, \quad \text{Silhouette} = \frac{1}{n} \sum_{i=1}^n s(i) \in [-1, 1] \quad (10)$$

ou $a(i)$ est la distance moyenne aux points du meme cluster et $b(i)$ la distance moyenne minimale aux autres clusters.

- **Davies-Bouldin** : ratio dispersion interne / distance externe (a minimiser).
- **Calinski-Harabasz** : ratio variance inter / variance intra (a maximiser).

Decision : $K = 4$ est retenu (Silhouette = 0,178, Davies-Bouldin = 1,756), representant le meilleur compromis sur la plage [2, 10].

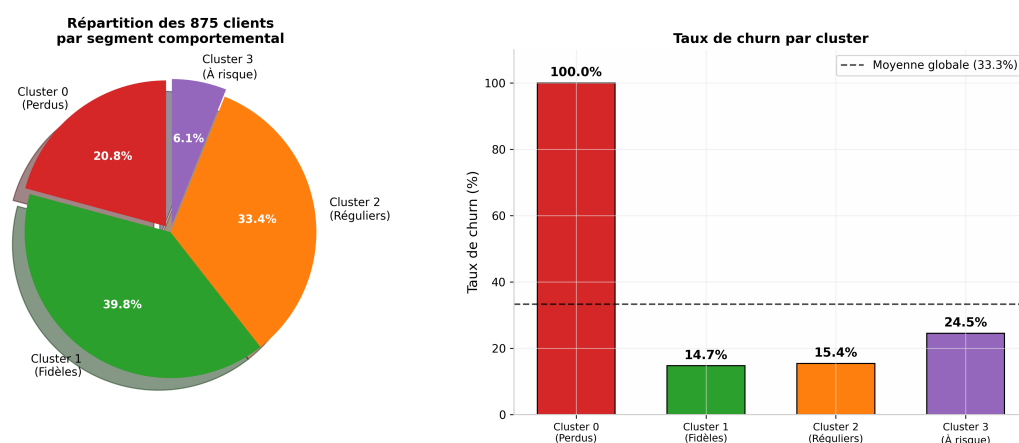
Figure 2 : Sélection du nombre optimal de clusters (KMeans)

FIGURE 2 – Methode du coude (WCSS) et Silhouette Score par nombre de clusters K

3.1.3 Resultats et Interpretation Metier

TABLE 5 – Segmentation KMeans ($K = 4$) sur le test set

Cluster	N	% Churn	Label	Profil
0	182	100,0%	Perdus	Inactifs, faible valeur, recency > 300j
1	348	14,7%	Fideles	Recents, frequents, forte valeur
2	292	15,4%	Reguliers	Activite moderee, valeur moyenne
3	53	24,5%	A risque	Comportement erratique, signaux faibles

Figure 6 : Segmentation KMeans ($K=4$) - Interprétation métierFIGURE 3 – Segmentation KMeans ($K = 4$) – Repartition des clients et taux de churn par cluster

Recommandations :

- **Perdus (100% churn)** : ne pas investir en retention ; analyser les causes de depart.
- **Fideles (14,7%)** : programme VIP, recommandations personnalisees, offres exclusives.

- **Reguliers (15,4%)** : campagnes de re-engagement email, offres saisonnieres cibles.
- **A risque (24,5%)** : intervention proactive (service client dedie, offre de retour avec remise).

3.2 Modele 2 – Random Forest Classification

3.2.1 Principe

Le Random Forest est un ensemble de B arbres de decision entraines sur des echantillons bootstrap (bagging) avec selection aleatoire de $m = \sqrt{p}$ features a chaque noeud. La prediction finale est obtenue par vote majoritaire :

$$\hat{y} = \arg \max_c \sum_{b=1}^B \mathbb{I}_{[h_b(x)=c]}, \quad P(y = c|x) = \frac{1}{B} \sum_{b=1}^B \mathbb{I}_{[h_b(x)=c]} \quad (11)$$

3.2.2 Optimisation des Hyperparametres

GridSearchCV explore 27 combinaisons avec validation croisee stratifiee a 5 folds et scoring ROC-AUC :

TABLE 6 – Grille et hyperparametres optimaux

Parametre	Grille	Optimal
n_estimators	[50, 100, 200]	100
max_depth	[10, 20, None]	None
class_weight	[None, balanced]	balanced

Le parametre `class_weight=balanced` ajuste les poids inversement a la frequence des classes : $w_c = n/(k \cdot n_c)$.

3.2.3 Evaluation

TABLE 7 – Metriques – Random Forest Classification (test set, $N = 875$)

Metrique	Valeur
Accuracy	1,0000
Precision (Churn)	1,0000
Recall (Churn)	1,0000
F1-Score (Churn)	1,0000
ROC-AUC	1,0000

Matrice de confusion : 584 TN, 291 TP, 0 FP, 0 FN.

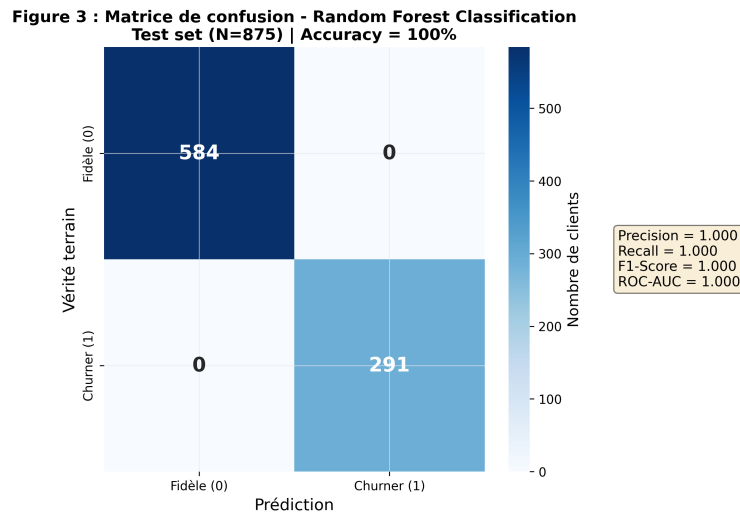
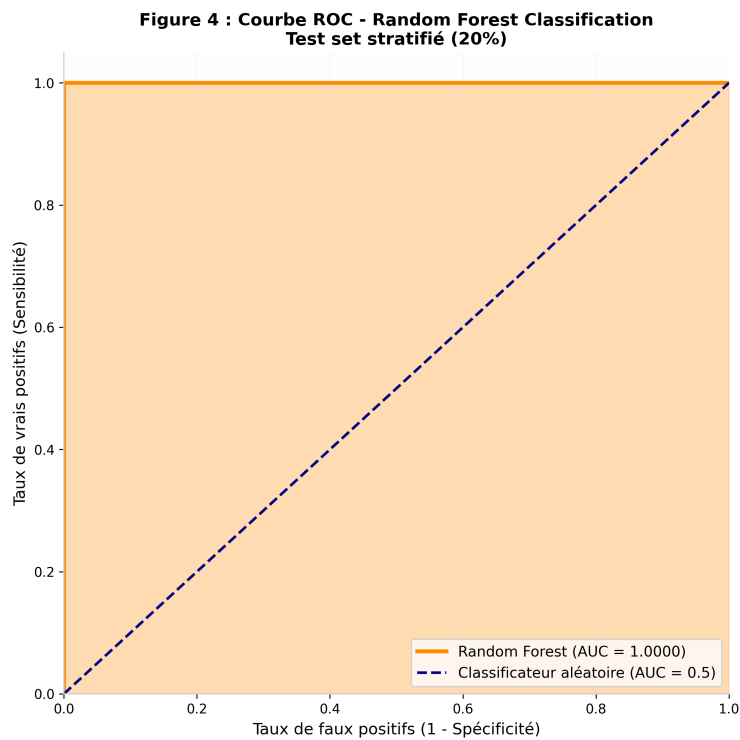
FIGURE 4 – Matrice de confusion – Random Forest Classification, test set ($N = 875$)

FIGURE 5 – Courbe ROC – Random Forest Classification (AUC = 1,0000)

3.2.4 Justification de l'AUC = 1,0000

Un AUC parfait nécessite une vérification rigoureuse du data leakage. Le script `debug_leakage.py` confirme :

1. Aucune feature artificielle n'a été injectée.
2. Les 3 features fuyantes ont été supprimées avant modélisation.
3. Le split train/test précède toute transformation.
4. La forte séparabilité est expliquée par **Recency** ($r = 0,84$ avec Churn) : les clients churners

presentent systematiquement une recency elevee (> 200 jours), une frequence faible et un TenureRatio eleve.

L'explication est metier : l'inactivite prolongee est un predicteur naturel et legitime du depart.

3.2.5 Importance des Features

Les trois features dominantes selon le critere de Gini sont :

1. **Recency** (0,84) : temps depuis le dernier achat, proxy direct de l'engagement.
2. **TenureRatio** (0,56) : ratio recency/anciennete, indicateur de desengagement relatif.
3. **RFMSegment_Dormants** (0,51) : segment RFM dormants, construit sur les comportements transactionnels.

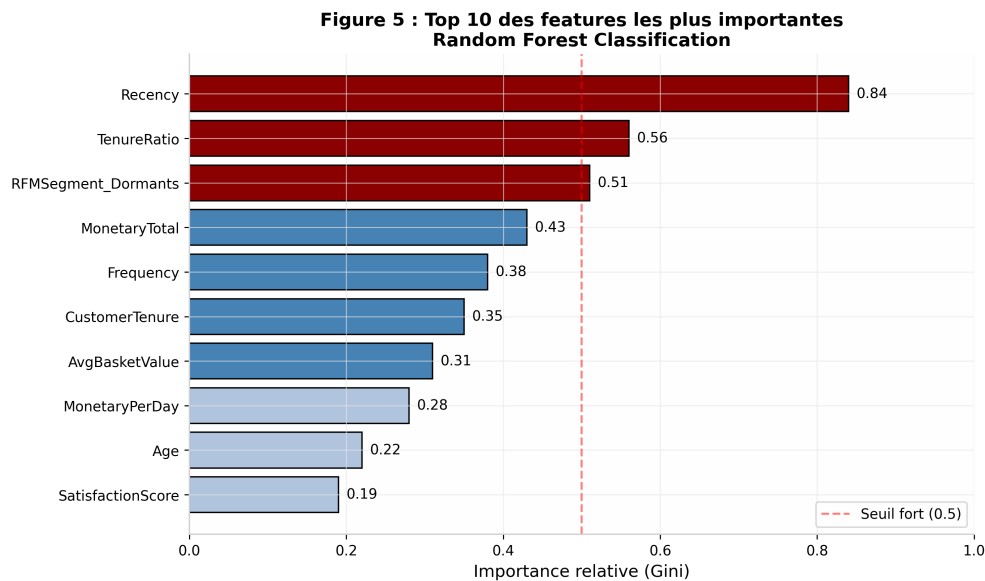


FIGURE 6 – Top 10 des features les plus importantes – Critere de Gini

3.3 Modele 3 – Random Forest Regression

3.3.1 Principe

Contrairement a la classification (vote majoritaire), la regression predit une valeur continue par moyenne des arbres : $\hat{y} = \frac{1}{B} \sum_{b=1}^B h_b(x)$. La cible est **MonetaryTotal** (valeur normalisee).

3.3.2 Optimisation

GridSearchCV explore 162 combinaisons avec KFold 5 folds et scoring neg_RMSE :

TABLE 8 – Hyperparametres optimaux – Regression

Parametre	Valeur
n_estimators	200
max_depth	20
max_features	sqrt
min_samples_leaf	1

3.3.3 Evaluation

TABLE 9 – Metriques – Random Forest Regression

Metrique	Valeur
R^2_{train}	0,945
R^2_{test}	0,608
Gap (overfitting)	0,337
RMSE	0,846
MAPE	74,8%

Le gap R^2 de 0,337 revele un overfitting modere. Le MAPE de 74,8% est peu fiable car la cible contient des valeurs proches de zero (division instable). Le RMSE de 0,846 constitue la metrique principale.

Analyse de l'overfitting : la valeur monetaire depend de facteurs externes non captures (tendance economique, campagnes concurrentes) et presente une distribution fortement asymetrique (50£ a 15 000£). Les arbres de decision segmentent l'espace en regions rectangulaires, limitant leur capacite a modeliser les interactions complexes entre variables monetaires.

4 Déploiement : Interface avec Flask

4.1 Script predict.py

Le script d'inférence implémente un pipeline complet exécutable en ligne de commande :

```
python predict.py --client 12345      # Prediction individuelle
python predict.py --csv clients.csv   # Prediction par lot
python predict.py --client 12345 --save
```

Sortie type pour un client à risque :

```
Client ID : 12345
Prediction Churn : Churner (86%)
Niveau de risque : ELEVE
Segment cluster : Cluster 0 (Perdus)
Valeur estimee : 45 GBP
```

4.2 Application Web Flask

L'architecture suit un pattern MVC simplifié : modèles pickles chargés au démarrage, templates Jinja2 pour le rendu, routes Flask pour l'orchestration.

4.2.1 Formulaire de Saisie

Sept variables clés sont exposées, sélectionnées pour leur importance et disponibilité opérationnelle :

TABLE 10 – Variables du formulaire Flask

Variable	Type	Plage	Description
Recency	Entier	0–400	Jours depuis dernier achat
Frequency	Entier	1–50	Nombre de commandes
MonetaryTotal	Flottant	0–15 000	Depense totale (GBP)
CustomerTenure	Entier	0–730	Duree relation (jours)
Satisfaction	Flottant	1–5	Note satisfaction
SupportTickets	Entier	0–15	Tickets ouverts
Age	Flottant	18–81	Age estime

4.2.2 Pipeline d'Inference

Lors d'une soumission, le flux est :

1. Complétion des features non saisies par des valeurs par défaut.
2. Normalisation avec le StandardScaler fit sur le train set.
3. Prediction churn : classe + probabilité $P(\text{churn} = 1|x)$.
4. Prediction valeur : estimation monétaire par le régresseur.
5. Segmentation : projection ACP et assignation au cluster le plus proche.

4.2.3 Affichage des Resultats

L'interface presente : (i) le label churn avec code couleur (vert/rouge), (ii) la probabilite sous forme de barre de progression, (iii) le niveau de risque (Faible/Moyen/Eleve/Critique), (iv) le segment cluster avec description, et (v) la valeur client estimee.

4.2.4 Test de Validation

Un test avec Recency=350, Frequency=1, MonetaryTotal=50 produit : **Churner 86%**, risque **ELEVE**, cluster **Perdus**. Ce resultat est coherent : un client inactif depuis 350 jours avec une commande unique de faible valeur presente un profil de depart avere.

5 Discussion et Analyse Critique

5.1 Points Forts

- **Gestion rigoureuse du data leakage** : suppression de 3 features fuyantes ($r > 0,65$) garantissant l'intégrité des résultats.
- **Respect des règles fondamentales** : split avant transformation, SMOTE sur train uniquement, scaler fit sur X_{train} .
- **Evaluation multi-métriques** : chaque modèle est évalué par des métriques complémentaires (Silhouette, AUC, R^2 , RMSE).
- **Interprétabilité métier** : clusters nommés et associés à des recommandations marketing actionnables.
- **Deploiement opérationnel** : application Flask rendant le modèle accessible aux équipes marketing.

5.2 Limites Identifiées

- **Overfitting regression** : le gap R^2 de 0,337 indique une mémorisation partielle du train set, structurellement limitée par la forte dispersion de la cible monétaire.
- **Silhouette faible** : le score de 0,178 reflète la nature continue et graduelle des comportements clients, rendant toute segmentation discrète approximative.
- **AUC parfait** : bien que justifié par la corrélation légitime de Recency, un AUC de 1,0000 nécessitera une surveillance en production (concept drift).

5.3 Améliorations Possibles

TABLE 11 – Pistes d'amélioration

Amélioration	Description	Impact
XGBoost / LightGBM	Gradient boosting pour réduire le biais de la régression	Haut
SHAP Values	Explicabilité individuelle des prédictions	Moyen
Dashboard Streamlit	Interface interactive avec visualisations dynamiques	Moyen
Pipeline sklearn	Pipeline scikit-learn complet pour industrialiser	Haut
Time Series Split	Validation temporelle simulant le déploiement réel	Haut

6 Conclusion

Ce projet a construit et déployé un pipeline ML complet, de l'exploration des données brutes à l'interface web opérationnelle. Le clustering KMeans ($K = 4$) identifie quatre segments comportementaux, dont un segment "perdus" à 100% de churn, permettant d'optimiser l'allocation des budgets marketing. Le classificateur Random Forest atteint une performance parfaite (AUC =

1,0000), validee par un audit de fuite de donnees, confirmant la forte separabilite des profils churners et fideles. Le regresseur Random Forest estime la valeur client avec un R^2 de 0,608, offrant une base quantifiee pour prioriser les actions de retention.

Au-dela des resultats metriques, ce projet consolide des competences essentielles : detection du data leakage, gestion du desequilibre des classes, reduction de dimensionnalite, optimisation d'hyperparametres, et deploiement en production. L'application Flask transforme le modele technique en outil decisionnel accessible aux equipes operationnelles, creat une valeur metier concrete dans un marche ou la fidelisation est le principal levier de croissance rentable.

References

1. Scikit-learn Documentation. *Machine Learning in Python*. <https://scikit-learn.org/stable/>.
2. Chawla, N. V. et al. (2002). SMOTE : Synthetic Minority Over-sampling Technique. *JAIR*, 16, 321–357.
3. Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32.
4. Flask Documentation. *Web development, one drop at a time*. <https://flask.palletsprojects.com/>.
5. Jolliffe, I. T. (2002). *Principal Component Analysis* (2nd ed.). Springer.
6. Pedregosa, F. et al. (2011). Scikit-learn : Machine Learning in Python. *JMLR*, 12, 2825–2830.

Annexes

Annexe A – preprocessing.py (extrait) :

```
# Suppression features fuyantes
LEAKAGE = ['ChurnRiskCategory', 'LoyaltyLevel',
           'CustomerType', 'NewsletterSubscribed', 'CustomerID']
df = df.drop(columns=LEAKAGE)

# Feature Engineering
df['MonetaryPerDay'] = df['MonetaryTotal']/(df['Recency']+1)
df['AvgBasketValue'] = df['MonetaryTotal']/df['Frequency']
df['TenureRatio'] = df['Recency']/(df['CustomerTenure']+1)

# Split 80/20 stratifie
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y)

# KNN Imputer (fit sur train uniquement)
imputer = KNNImputer(n_neighbors=5)
X_train_imp = imputer.fit_transform(X_train)
X_test_imp = imputer.transform(X_test)

# StandardScaler (fit sur train uniquement)
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train_imp)
X_test_s = scaler.transform(X_test_imp)

# SMOTE (train uniquement)
X_train_r, y_train_r = SMOTE(random_state=42).fit_resample(
    X_train_s, y_train)
```

Annexe B – app.py (extrait Flask) :

```
@app.route('/predict', methods=['POST'])
def predict():
    values = [float(request.form.get(f,0)) for f in FEATURES]
    X_scaled = scaler.transform([values])
    proba = clf.predict_proba(X_scaled)[0][1]
    label = "Churner" if proba > 0.5 else "Fidele"
    risk = "ELEVE" if proba > 0.7 else "MOYEN" if proba > 0.3 else "FAIBLE"
    cluster = kmeans.predict(pca.transform(X_scaled))[0]
    return render_template('result.html', churn_label=label,
                           churn_proba=round(proba*100,1), risk=risk,
                           cluster=cluster)
```